Introduction to Object-Oriented Programming (OOP) in Java

A beginner-friendly guide to the four pillars of OOP

---

1. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects" — units that bundle together data (fields/attributes) and behavior (methods/functions). Java is a fully object-oriented language (except for primitive types), meaning almost everything in a Java program is modeled as an object or a class.

The main goals of OOP are to make code more modular, reusable, easier to maintain, and closer to how we naturally think about real-world entities (like a Car, a Student, or a BankAccount).

2. Classes and Objects

A **class** is a blueprint or template that defines the structure (fields) and behavior (methods) that its objects will have. An **object** is a concrete instance created from that class, with its own actual values stored in memory.

Example:

```
class Car {
  String color;
  int speed;

  void drive() {
    System.out.println("The car is driving.");
  }
}

public class Main {
  public static void main(String[] args) {
    Car myCar = new Car(); // object created
    myCar.color = "Red";
    myCar.drive();
  }
}
```

3. The Four Pillars of OOP

Java's object-oriented model rests on four fundamental principles: Encapsulation, Inheritance, Polymorphism, and Abstraction.

3.1 Encapsulation

Encapsulation means bundling data (fields) and the methods that operate on that data into a single unit (a class), and restricting direct access to some of an object's components. In Java, this is achieved using access modifiers such as `private`, `protected`, and `public`, along with getter and setter methods.

```
class BankAccount {
  private double balance;

  public double getBalance() {
    return balance;
  }

  public void deposit(double amount) {
    if (amount > 0) balance += amount;
  }
}
```

Here, `balance` cannot be changed directly from outside the class — it can only be modified through controlled methods like `deposit()`. This protects the object's internal state.

3.2 Inheritance

Inheritance allows one class (the subclass/child) to acquire the fields and methods of another class (the superclass/parent) using the `extends` keyword. It promotes code reuse and establishes an "is-a" relationship.

```
class Animal {
  void eat() {
    System.out.println("This animal eats food.");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("The dog barks.");
  }
}

// Dog objects can use both eat() and bark()
Dog d = new Dog();
d.eat();
d.bark();
```

3.3 Polymorphism

Polymorphism means "many forms" — the ability of an object to take on different behaviors depending on context. Java supports two types:

- **Compile-time (Method Overloading):** Multiple methods with the same name but different parameter lists within the same class.
- **Runtime (Method Overriding):** A subclass provides its own implementation of a method already defined in its superclass, using the same signature.

```
class Animal {
  void sound() {
    System.out.println("Some generic animal sound");
  }
}

class Cat extends Animal {
  @Override
  void sound() {
    System.out.println("Meow");
  }
}

Animal a = new Cat(); // reference type Animal, object type Cat
a.sound(); // prints "Meow" (runtime polymorphism)
```

3.4 Abstraction

Abstraction means hiding complex implementation details and exposing only the essential features of an object. In Java, this is achieved using **abstract classes** (declared with the `abstract` keyword) and **interfaces**.

```
abstract class Shape {
  abstract double area(); // no implementation here
}

class Circle extends Shape {
  double radius;
  Circle(double radius) { this.radius = radius; }

  double area() {
    return Math.PI * radius * radius;
  }
}
```

4. Summary Table

Concept	Keyword(s) in Java	Purpose
Encapsulation	private, public, protected	Protect and control access to data
Inheritance	extends	Reuse code, build class hierarchies
Polymorphism	@Override, overloading	Same interface, different behavior
Abstraction	abstract, interface	Hide implementation details

5. Why OOP Matters

- **Modularity:** Code is organized into self-contained classes.
- **Reusability:** Existing classes can be extended instead of rewritten.
- **Maintainability:** Changes to one class have minimal impact on others.
- **Scalability:** Large, complex systems are easier to manage.

This document covered the core OOP concepts in Java: classes, objects, encapsulation, inheritance, polymorphism, and abstraction — the foundation for writing well-structured Java programs.